MOTION CARE (REALIDAD AUMENTADA Y MIXTA)

Pablo Álvarez de Lara
Manuel Alejandro Villalba
Hilario Javier Del Valle
Olaia Picabea

26 de mayo de 2026

CONTENIDOS

1. Idea general 1
2. Motivación y objetivos 1
3. Desarrollo 2
3.1. Estructura global 2
3.2. Estructura individual 5
3.2.1. Tutorial (Manuel Alejandro) 6
3.2.2. Comedor (Olaia Picabea) 7
3.2.3. Sala de estar (Hilario Javier Del Valle Escolar) 10
3.2.4. Cocina (Pablo Álvarez de Lara Fernández) 12
4. Conclusiones 15
5. Trabajos futuros 16
6. Bibliografía 18

1. Idea general

Nuestra idea principal era la de generar una experiencia de realidad virtual compuesta por una serie de minijuegos diseñados para apoyar la rehabilitación y estimulación de las personas, especialmente ancianas con dificultades psicomotrices.

El proyecto busca ofrecer una alternativa accesible a las terapias tradicionales, haciendo uso de ejercicios físicos y cognitivos que fomentan la participación del usuario. La experiencia pretende mejorar habilidades como la coordinación motora, los reflejos, la orientación espacial y la movilidad de las extremidades superiores, todo ello en un entorno seguro, inmersivo y entretenido que ayude a incrementar la motivación y el bienestar emocional de los usuarios.

Para crearla, decidimos hacer uso de los conocimientos obtenidos del máster y utilizar Unity, aprovechando sus herramientas especializadas para la creación de entornos de realidad virtual interactivos y optimizados. Gracias a este motor gráfico, ha sido posible implementar sistemas de interacción intuitivos, físicas adaptadas y mecánicas accesibles para usuarios de edad avanzada, priorizando siempre la facilidad de uso y la comodidad durante la experiencia. Además, el uso de Unity ha permitido integrar animaciones, recolección de datos y sistemas

de seguimiento de movimiento que facilitan la adaptación de los minijuegos a distintos niveles de capacidad psicomotriz, ofreciendo así una experiencia flexible y personalizable orientada tanto al entretenimiento como al apoyo terapéutico.

A nivel artístico, propusimos un estilo visual “cartoony” y suave, caracterizado por el uso de colores cálidos, formas simples y entornos amigables que transmiten tranquilidad y cercanía. Esta dirección artística tiene como objetivo el generar una sensación de comodidad y reducir la sobrecarga visual, algo especialmente importante para usuarios de edad avanzada. Los escenarios y elementos interactivos presentan diseños fácilmente reconocibles y agradables a la vista, favoreciendo la orientación y comprensión de las actividades dentro del entorno virtual. De esta manera, el apartado visual no solo cumple una función estética, sino que también contribuye a mejorar la accesibilidad, la inmersión y la conexión emocional de los usuarios con la experiencia.

2. Motivación y objetivos

Motion Care nace a partir de la idea de intentar llevar el ejercicio físico y la rehabilitación de diversas aptitudes a la mayor cantidad de gente posible. Con un especial enfoque a personas de la tercera edad, ya que en una mayoría de casos se trata de gente que se beneficiaría ampliamente de este ejercicio.

Como exponen Frontera y Walter en su paper de 2018[1], es innegable que la población mundial está envejeciendo, llegando cada vez a esperanzas de vida más longevas. Esto supone un nuevo reto ya que, junto a esto, también llega una nueva etapa de deterioro corporal. Durante esta fase, las personas son más proclives tanto a accidentes como a enfermedades, aumentando progresivamente su gravedad.

Debido a esta problemática, hace falta buscar nuevas soluciones adaptadas al mundo moderno en el que vivimos. Uno de los grandes dilemas actuales que enfrentan las personas de la tercera edad no es la falta de posibilidades de ejercicio, sino la habilitación de espacios buenos, seguros y monitoreados para el desarrollo de este. A menudo, estos espacios están desactualizados, cuentan con máquinas antiguas y poco seguras; y suelen no tener vigilancia, por lo que las lesiones pueden producir situaciones complicadas y peligrosas. Una solución es el uso de nuevas tecnologías como la realidad mixta (XR) para facilitar el acceso de la gente de tercera edad a formas de ejercicio, tanto mentales como físico, en espacios más controlados y sin necesidad de requerir espacios especializados. Una simple aplicación, con unas gafas de la mano de un acompañante, puede convertir una habitación de casa en un trepidante juego que no solo estimule el cuerpo sino la mente y la diversión también.

En el paper mencionado antes, se explica la importancia del ejercicio físico dentro de una buena salud, centrándose en el ejercicio de alto rendimiento (correr, montar en bici, etc.). En el caso de Motion Care, el objetivo no es llegar a ese nivel de exigencia física sino proponer una forma agradable y familiar para el usuario de iniciar una rutina de ejercicios. Por ello, la base de la aplicación es el trabajo en ámbitos menos exigentes, pero no menos importantes, como la motricidad, la coordinación y la memoria. Con intención de plantear una base recia para aumentar paulatinamente la cantidad e intensidad de ejercicio.

Cabe destacar también, que, a pesar de esta focalización en la tercera edad, la aplicación se ha pensado como apta para cualquier persona independientemente de la edad. Al fin y al cabo, un buen estado físico y mental es fundamental para un buen desarrollo vital. Por lo que el desarrollo se ha manejado con esto en mente.

3. Desarrollo

3.1. Estructura global

El desarrollo de la aplicación se ha planteado siguiendo una metodología de trabajo colaborativa y modular, en la que cada integrante del equipo ha asumido la responsabilidad del diseño e implementación de una estancia concreta de la vivienda virtual. La estructura general del proyecto se divide en cuatro áreas principales: el salón, la cocina, el comedor y un tutorial inicial. Esta distribución de tareas, posible gracias a la creación y trabajo por ramas de un repositorio de GitHub, ha permitido organizar el trabajo de manera más eficiente, facilitando el desarrollo simultáneo de distintas partes de la aplicación y mejorando la coordinación entre los miembros del equipo.

El flujo general de la aplicación sigue una estructura secuencial diseñada para guiar al usuario de forma progresiva a través de la experiencia en realidad virtual. La interacción comienza con un tutorial inicial donde se enseñan las mecánicas básicas y el uso de los controles, facilitando la adaptación al entorno virtual. Tras completarlo, el usuario accede al comedor, continúa posteriormente por la sala de estar y finaliza la experiencia en la cocina, completando diferentes pruebas correctamente guiadas y con un feedback preciso.

Con el objetivo de facilitar la integración y organización del trabajo realizado por cada integrante del equipo, y así poder realizar cambios a tiempo real sin colapsar el trabajo de los demás, cada estancia de la aplicación se ha desarrollado como un prefab independiente que agrupa todos los elementos, objetos y mecánicas necesarios de su correspondiente apartado. Al iniciar la aplicación, todos estos prefabs permanecen desactivados, permitiendo una gestión más eficiente de los recursos y del flujo de la experiencia. La activación y desactivación de cada prueba se controla mediante un GameManager central, cuyo script asociado se encarga de gestionar la transición entre zonas siguiendo el orden establecido dentro de la aplicación.

Este enfoque ha permitido realizar una integración conjunta de forma eficiente, ya que todos los prefabs de las distintas estancias se desarrollaron a partir de un prefab base compartido que incorporaba un cubo utilizado como referencia espacial. Gracias a ello, todas las zonas mantuvieron las mismas dimensiones, escalas y posiciones de referencia dentro del entorno virtual, lo que facilitó considerablemente la unificación de las diferentes partes del proyecto y simplificó el proceso de integración final. Una vez desarrolladas las pruebas, la colocación de las mismas fue directa y sencilla, y permitieron añadir detalles como la modificación de escalas en forma de transición, para así dar un resultado más satisfactorio.

Como se menciona al inicio, la idea principal es la de analizar la interacción de las personas con la simulación, y así ayudarlas con posibles dificultades psicomotrices. Para ello, es necesario estudiar dichas interacciones a fondo, por lo que en cada estancia se registran métricas específicas orientadas a evaluar las capacidades que se buscan estimular en el usuario, como la memoria, el movimiento o la capacidad de secuenciación. Para gestionar esta información se ha implementado un DataManager, diseñado bajo el patrón singleton para garantizar su acceso global desde todos los minijuegos. Este sistema centraliza la recopilación de datos del usuario, el seguimiento de las sesiones y el registro de las tareas realizadas, generando finalmente un archivo en formato JSON que almacena de manera estructurada toda la información obtenida durante la experiencia. Para la recopilación de fechas y tipos de tareas se ha utilizado las estructuras de datos de Date y enum propias de Unity, y debido a que se necesita información de tiempo, se emplea un ChronometerManager, también implementado como singleton, que permite a los minijuegos registrar los tiempos de ejecución de cada actividad. Estos sistemas garantizan una recogida de datos consistente, unificada y accesible desde cualquier parte de la aplicación.

3.2. Estructura individual

3.2.1. Tutorial (Manuel Alejandro)

El tutorial de la aplicación de realidad virtual ha sido diseñado como una fase inicial de adaptación e introducción a las mecánicas principales del entorno VR. El enfoque está especialmente orientado a personas mayores, por lo que todas las pruebas presentan acciones cotidianas y fácilmente reconocibles, reduciendo así la sensación de complejidad tecnológica y facilitando la inmersión en el entorno virtual. Se compone de tres pruebas diferenciadas, cada una enfocada en una mecánica concreta. Las actividades han sido planteadas siguiendo una progresión gradual, permitiendo que el usuario gane confianza mientras aprende el funcionamiento básico del sistema.

Para ello, el motor hace uso del script TutorialHandler, el cual se encarga de organizar y gestionar las diferentes pruebas del tutorial. Obtiene una lista de todos los grupos de objetos que componen cada reto, y los activa y desactiva en función de la necesidad del usuario. Para asegurar una transicion suave y cómoda, utiliza Tweening y hace que las escalas de los objetos disminuyan y aumenten de forma suave y gradual, lo cual, combinado con feedback sonoro, genera una experiencia mucho más accesible.

La primera prueba consiste en coger una manzana y arrastrarla hasta un bol que contiene más manzanas. Esta actividad introduce al usuario en la mecánica de selección y manipulación de objetos virtuales. Al tratarse de un objeto cotidiano y una acción sencilla, se busca generar una experiencia cómoda y natural desde el primer contacto con la aplicación.

Para lograr el funcionamiento de la prueba, hay dos pilares fundamentales presentes. El primero es el elemento apple, que hace uso de las funcionalidades de Grabbable del paquete de VR de Unity para lograr que el usuario la agarre y la manipule por el entorno. El otro componente fundamental es el de PlateDetector, que implementa una funcionalidad Trigger encargada de detectar la colisión entre el plato y la manzana. Una vez detectada, activa una animación suave de colocación de la manzana en el plato y da paso a la siguiente prueba.

La segunda prueba aumenta ligeramente la complejidad de la interacción. En este caso, el usuario debe recoger varios libros y colocarlos dentro de una caja. A diferencia de la actividad anterior, esta tarea requiere repetir movimientos de agarre y colocación varias veces, reforzando así el aprendizaje del funcionamiento de la aplicación, mejorando la fluidez y confianza dentro del entorno virtual. El diseño de esta prueba también busca fomentar movimientos amplios y controlados de brazos y manos, contribuyendo al trabajo de movimiento de manera funcional y accesible.

Para lograr el resultado, se utiliza una metodología muy similar a la de la anterior prueba. Cada uno de los libros que el jugador se encarga de tomar y colocar en la caja implementa las funcionalidades Grabbable del paquete VR de Unity. Para detectarlos, la caja dispone de un BookBoxDetector, que de nuevo funciona a modo de Trigger y detecta cuando cualquiera de los libros entra en su rango de detección y los almacena en su interior usando una interpolación de escala y movimiento. Todos los elementos de importancia del tutorial hacen uso del componente Outline, el cual se encarga de proporcionar un reborde de color a los objetos 3d para resaltarlos y que sea más fácil reconocerlos a primera vista.

La tercera y última prueba introduce una mecánica diferente: la interacción mediante pulsación. El usuario debe acercarse a un botón y presionarlo para encender una televisión virtual. Con esta actividad se enseñan acciones de señalamiento y activación de elementos interactivos dentro del entorno. Aunque es una tarea aparentemente simple, requiere precisión en el movimiento y comprensión dentro del sistema virtual. La respuesta inmediata de la televisión al encenderse ayuda al usuario a comprender claramente el funcionamiento de la interacción y refuerza la sensación de control sobre el entorno.

Para lograr esta interacción con el botón, se hace uso de la funcionalidad PokeInteractable de Unity, que permite detectar la pulsación con un dedo del jugador en un área correspondiente. Al detectar dicha colisión, la simulación reproducirá una música a modo de recompensa con el componente AudioSource, así como unas partículas de notas de sonido realizadas mediante el ParticleSystem de Unity. Ambos componentes están diseñados para no ser muy invasivos y que resulten un tiempo de relajación para el jugador, programadas para loopearse constantemente hasta que el programa transicione a la primera prueba real.

Todas las pruebas incorporan un sistema de feedback sonoro claramente definido. Cada acción correcta genera una respuesta auditiva específica, permitiendo que el usuario reciba confirmación inmediata de que ha realizado la tarea adecuadamente. Este refuerzo auditivo cumple varias funciones importantes: mejora la comprensión de las mecánicas, aumenta la sensación de logro, facilita la orientación dentro del entorno virtual y aporta seguridad durante la experiencia. En usuarios de edad avanzada, este tipo de retroalimentación resulta especialmente útil para reducir la incertidumbre y favorecer una interacción más intuitiva y motivadora.

Al utilizar tareas simples relacionadas con actividades cotidianas, se minimiza la barrera tecnológica y se facilita que personas con poca experiencia previa en realidad virtual puedan desenvolverse con mayor comodidad. La estructura progresiva del tutorial permite que el usuario aprenda de forma natural, avanzando desde acciones básicas de manipulación hasta interacciones más específicas. En conjunto, el tutorial actúa como una herramienta de introducción tanto al entorno VR como a los ejercicios planteados por la aplicación. A través

de actividades sencillas, feedback inmediato y mecánicas intuitivas, se busca crear una experiencia accesible y comprensible, que prepare al usuario para utilizar el resto de funcionalidades del programa con confianza y autonomía.

3.2.2. Comedor (Olaia Picabea)

En esta primera fase se le pide al usuario que coloque la mesa. La práctica se divide en 3 partes - fácil, medio y difícil – aunque la tarea principal sea la misma. La meta de esta parte es trabajar la motricidad fina, obligando al usuario a controlar movimientos pequeños y precisos con las manos. Por otro lado, en la parte difícil de la prueba, se pide al usuario que se acuerde de donde tiene que dejar los cubiertos, por tanto, también se trabaja la memoria en segundo plano.
Mi desarrollo se ha llevado a cabo en el prefab Comedor y su estructura general es la siguiente:

- DificultadFacil: Game object vacío que incluye modelos, scripts etc. Para el correcto funcionamiento del nivel fácil.
- DificultadMedia: Game object vacío igual que el primero, pero con los cubiertos reposicionados de una manera menos intuitiva para el usuario.
- DificultadDificil: Igual que los anteriores, pero sin los dibujos de las guías sobre la mesa. Además, en esta parte, cuando se termina de colocar los cubiertos, se enseña una botella que el usuario tiene que utilizar para rellenar la copa.
- Cube: Cubo de la mesa.
- PlacementManager: Manager para gestionar el posicionamiento de los cubiertos. La funcionalidad principal es recolocar los cubiertos en sus posiciones exactas y hacer que deje de ser interactable.
- GameManager: Manager general del comedor, donde se gestiona el cambio de minijuego y la reproducción de distintos audios.
- AudioSources: Game object vacío que contiene como hijos todos los audios sources que hacen falta en esta parte del comedor.

MINIJUEGOS

Minijuego 1: Dificultad fácil
Este primer minijuego se considera la variación más básica, ya que los cubiertos aparecen cerca de sus respectivas guías, y, por tanto, el usuario no tiene que desplazar mucho el objeto.
Cada modelo por colocar contiene 3 scripts: ReleaseDetector (que gestiona el momento que se suelta el objeto, para luego posicionarlo en su slot), TableObject (que contiene una función que elimina su propiedad interactuable) y RespawnWhenFarComedor (script que devuelve el objeto a la mesa si se cae). Además de estos scripts personalizados, todos los objetos interactuables contienen grababble, GrabInteractable y HandGrabInteractable.
Además de estos objetos, encontramos unas guías sobre la mesa. Para crearlas, simplemente dibujé un outline y lo convertí en material, para luego asignarlos a planos.
Como se ha comentado antes, en las guías se incluyeron unos slots (o puntos de posición) para colocar los objetos y para que el usuario sintiera que lo ha hecho bien. Esos slots contienen un script llamado PlacementSlot, que simplemente asigna un identificador y jun radio de margen.

Minijuego 2: Dificultad media

El minijuego 2 funciona exactamente igual. La única diferencia es que la posición inicial de los cubiertos es menos intuitiva para el usuario, es decir, los objetos no se encuentran cerca de sus guías.

Minijuego 3: Dificultad dificil

La estructura de esta parte es muy parecida a la anterior. La mayor diferencia es que no se han incluido las guías visuales (aunque se han mantenido los slots).
Cuando es usuario coloque los cubiertos, aparece una botella que tendrá que utilizar para rellenar una copa. Para ello se ha desarrollado un script BottlePour que se encarga de enseñar las gotas de agua dependiendo del ángulo de la botella. Para crear el efecto de que la copa se está llenando, he añadido un PourSpace. Este game object contiene un capsule collider que se activa cuando aparece la botella en la escena. Además, tiene un script CupVolume que se encarga de calcular cuánto liquido ha caído dentro, y cuanto ha caído fuera. Por último, para dar el efecto visual del líquido, he insertado un cilindro que va subiendo, dependiendo del porcentaje de agua que ha entrado en la copa. Para gestionar eso se utiliza el script LiquidVisual.

SCRIPTS DESARROLLADOS

Todos los scripts de esta parte están dentro de la carpeta “Assets/Scripts/Comedor”.

- BottlePour: script que se encarga de spawnear las gotas cuando el ángulo de la botella supera los 45 grados. Además, controla cuantas gotas se han caído para parar cuando llegue a 300. Cuando el agua se termina, llama a un manager que se encargará de guardar la métrica en el json.
- CupVolume: este código se encarga de sumar la cantidad de gotas que caen dentro de la copa. También elimina las instancias de las gotas de la escena.
- Drop: script que elimina el game object de la gota si baja de
- Una altura de –0.5.
- GameManagerComedor: game manager de la escena del comedor. Este se encarga de pasar al siguiente minijuego cuando se terminan de colocar los cubiertos y de reproducir los audios de la escena.
- LiquidVisual: script que sube el cilindro que simula el agua en la copa teniendo en cuenta el porcentaje de llenado que se calcula en CupVolume.cs.
- PlacementManager: script que coloca los subiertos en su slot correspondiente si el usuario lo ha dejado suficientemente cerca de este.
- PlacementSlot: código para identificar cada slot con su id y radio aceptado correspondiente.
- ReleaseDetector: script que detecta si el cubierto ha sido soltado, para así detectar su posición en PlacementManager y colocarlo en su slot.
- TableObject: script que desactiva los componentes de interacción de los cubiertos una vez se han colocado.

RETOS DE ESTA SALA

El mayor reto fue dejar de lado OpenXR y adaptarme al sistema de Meta SDK. Aun así, conseguimos entenderlo y crear funcionalidades que al principio no sabíamos hacer.
Por otro lado, un reto importante fue adaptar la escena y aplicación a un usuario totalmente diferente a lo que estábamos acostumbrados. Eso conllevó ponernos en la piel de nuestros abuelos y abuelas y analizar si entenderían la experiencia como nos gustaría. Ese ha sido un tema que hemos tenido en mente durante todo el proceso.
El último reto que viví, un poco más técnico, fue desarrollar la interacción del usuario con la botella, así como la acción de verter el líquido en el vaso y que se viera físicamente realista. Tuve muchos problemas con los colliders, la gravedad, el cálculo del porcentaje... pero al final todo se solucionó y se creó una sala estéticamente bonita, clara y funcional.

3.2.3. Sala de estar (Hilario Javier Del Valle Escolar)

Mi aportación al proyecto consiste en el diseño e implementación de dos minijuegos orientados a la estimulación de la memoria dentro de la Sala de estar. Ambos minijuegos Simon Dice y Emparejar Cartas se integran en un único prefab principal (SalaDeEstar) y se activan o desactivan dinámicamente según el estado general del juego gestionado por el GameManager.

Todo el contenido desarrollado se encuentra organizado en la carpeta Sala de estar, que incluye:

- AudioClips: clips de voz y efectos utilizados en ambos minijuegos.
- JPEG: imágenes empleadas en las cartas.
- Materials: materiales aplicados a los modelos.
- Prefabs: prefabs completos de Simon Dice y Emparejar Cartas.
- Scripts: divididos en Effects, Enums y Managers.
- Escena y prefab principal con toda la lógica integrada.

El prefab SalaDeEstar contiene los siguientes elementos principales:

- Cube (trablero donde se desarrollan las actividades).
- ParejaCartas: contenedor de todos los prefabs de cartas.
- SimonDice: contenedor de libros y tazas del minijuego.
- AudioManager, ParejaCartasManager, SimonDiceManager y GameManager, responsables de la lógica y coordinación de la escena.

MINIJUEGOS

Minijuego 1: Simon Dice

Los prefabs de este minijuego incluyen 4 libros y 4 tazas, obtenidos del paquete LowPolyLivingRoomPack. Cada objeto incorpora:

- Scripts de interacción de Meta: Grabbable, HandGrabInteractable, InteractableUnityEvent.
- Scripts propios: AppearEffect, BlinkEffect, FadeEffect, ReinicioCaidaManager e InteractiveObject (que contiene la función `OnCogido()`).

En este caso no fue posible separar la interacción en un objeto hijo, ya que interfería con el funcionamiento del HandGrabInteractable.

El tweening de AppearEffect reduce temporalmente la escala del objeto a 0, lo que provocaba deformaciones al agarrarlo. Para evitarlo, los objetos se instancian inicialmente en un HiddenPoint con su escala real y posteriormente se trasladan a la mesa.

Minijuego 2: Emparejar Cartas

Cada carta es un prefab compuesto por:

- Objeto padre:
  - Rigidbody, BoxCollider, MeshRenderer.
  - Scripts propios: AppearEffect, TurnEffect, FadeEffect2, CardInteractable.
- Hijo “Interaction”:
  - PokeInteractable, InteractableUnityEvent, PointableElement, Clipped Plane Surface, Plane Surface, Bounds Clipper.

La interacción se realiza colocando el dedo índice sobre la carta, lo que activa el evento `OnTouched()` definido en CardInteractable. Este evento comunica con el ParejaCartasManager, que gestiona el volteo, la comprobación de parejas y el flujo del minijuego. Las cartas han sido modeladas por mí en 3ds Max, adaptadas para funcionar correctamente con los sistemas de interacción de Meta XR.

SCRIPTS DESARROLLADOS

Effects

- AppearEffect: aparición progresiva mediante tweening.
- BlinkEffect: parpadeo visual para guiar al usuario durante la secuencia de Simon Dice.
- FadeEffect: desaparición suave del objeto.
- TurnEffect: giro y recolocación de las cartas al ser seleccionadas.

Enums

Definen los tipos de cartas, clips, libros y tazas, permitiendo una identificación clara desde los managers.

Managers

- SimonDiceManager: controla niveles de dificultad, secuencias, validación de acciones y tiempos.
- ParejaCartasManager: gestiona el volteo, comparación y reinicio del minijuego de cartas.
- ReinicioCaidaManager: reposiciona automáticamente objetos que caen fuera de la mesa.
- GameManager: coordina el inicio y final de cada minijuego dentro de la escena.

RETOS DE ESTA SALA

Los mayores retos del proyecto han sido:

- Integrar correctamente los scripts de interacción de Meta XR, ya que requieren una combinación muy específica de componentes para funcionar. La ausencia o mala configuración de uno solo impedía la interacción.
- Pulir detalles que marcan la diferencia, como tweenings, efectos visuales, estructura modular del código y generación de audios adecuados.
- Garantizar una experiencia fluida, con objetos que aparecen, desaparecen, se iluminan o se reposicionan según el estado del juego.

3.2.4. Cocina (Pablo Álvarez de Lara Fernández)

MINIJUEGO

Para esta sala, se presenta una cocina que envuelve ligeramente al usuario sin necesidad de quitarle la visión del espacio real, intentando mantener la presencia sin extraer del todo al usuario de su orientación. Esto se ha decidido así al ser esta la última de las salas en reproducirse, cuando el usuario ya se ha acostumbrado levemente al ambiente virtual.

Además, esto ayuda a aumentar el compromiso con la aplicación, favoreciendo la vuelta del usuario y su progreso en las aptitudes trabajadas en ella.

Dentro de esta sala, el reto es montar una serie de tres bocadillos siguiendo una receta. Esta receta solo se muestra temporalmente antes de que aparezcan los ingredientes, trabajando la memoria factual. Una vez aparecen los ingredientes, el usuario debe colocarlos uno a uno posicionándolos encima del anterior, trabajando en su coordinación gruesa. A parte de esto, los diferentes ingredientes aparecen esparcidos por la encimera y la estantería, forzando al usuario a moverse en rangos cada vez más amplios, lo que ayuda a mejorar su movilidad. Estos rangos se modifican según los resultados obtenidos en la ronda anterior, incrementando la dificultad si en la ronda anterior lo ha hecho bien y rápido.

El flujo de cada ronda queda definido como se muestra a continuación:

1. El usuario coloca el plato vacío en la tabla amarilla (delante de él).
2. Aparece la receta en la pizarra blanca (delante de la tabla amarilla) durante 20 segundos.
3. Pasado el tiempo, desaparece la receta y aparecen los ingredientes.
4. El usuario coloca los ingredientes uno encima del otro sobre el plato.
5. Una vez considera que el bocadillo está completo, coloca el plato con los ingredientes encima de la tabla verde (a su izquierda).
6. Se vuelve a mostrar la receta y el resultado (aciertos, fallos y puntuación) en la pizarra blanca.
7. Si se ha resuelto sin fallos y en menos de 40 segundos, se incrementa la dificultad (tres niveles diferentes).

Además de esto, la sala cuenta con un apartado artístico trabajado, incluyendo modelos adecuados y sonido para dar tanto las instrucciones como retroalimentación sobre las acciones del usuario.

Para la parte visual, se han utilizado modelos provenientes de paquetes de la tienda de Unity para el mobiliario de cocina (encimera, extractor, tablas y estanterías). Para la sonora, se han utilizado efectos de internet sin derechos y audios creados a partir de IA sin derechos a través de la página de ElevenLabs[2].

SCRIPTS DESARROLLADOS

Para implementar esta funcionalidad se ha usado un sistema de sciprts provenientes tanto de la SDK de Meta como implementados por el propio desarrollador.

Principalmente, la sala se basa en el uso de Snap Interactors e Interactables de la SDK. Esto permite entender los objetos como “llaves y cerraduras” que encajan unas con otras. Además, se han utilizado Tag Sets y Tag Set Filters para restringir los objetos que pueden colocarse en un sitio. Por ejemplo, solo el plato puede colocarse sobre las tablas.

También se han utilizado los scripts correspondientes (visibles desde el wizard de Meta en Unity o en los tutoriales oficiales de Meta) para implementar una mecánica de agarre que se aplica al plato y los ingredientes.

Por otro lado, los scripts implementados por el desarrollador son los siguientes:

- KitchenMingameManager.cs: El controlador principal de la sala que gestiona el flujo de la aplicación y la llamada a las funciones necesarias de otros scripts.

- Scripts relacionados con los snaps:
    - GrabbableSocketController.cs: Desactiva y activa los componentes del agarre para que, una vez colocado un ingrediente encima de otro, el de abajo no se pueda agarrar sin quitar el de arriba.

    - StartRoundSocketController.cs: Llama a la función de inicio de ronda del manager, utilizando el snap asociado como lanzamiento del evento.

    - RecipieCheckSocketController.cs: Llama a la función de comprobación de receta del manager, utilizando el snap asociado como lanzamiento del evento.

- Scripts asociados con los bocadillos:
    - SandwichManager.cs: Encargado de manejar la estructura de datos del bocadillo que el usuario monta, añadiendo y quitando los ingredientes que se colocan en él.

    - ComparisonResult.cs (no MonoBehaviour): Clase que sirve como estructura de datos para almacenar los resultados finales de la comparación entre la receta y el bocadillo. Contiene una función para guardar el resultado en el JSON utilizando la clase DataManager.

    - ItemController.cs: Implementa dos enumerados para manejar tipo y categoria de los ingredientes. Ademas, realiza las llamadas pertinentes a la clase SandwichManager para añadirse a la lista del bocadillo si se pone encima del último ingrediente o quitarse si se mueve de esa posición. Además, se suscribe al evento de final de ronda del manager para destruirse si se esta se finaliza.

- Otros scripts:
    - MinigameAudioManager.cs: Maneja los diferentes fragmentos de audio, reproduciéndolos cuando se llama a sus funciones. Estos se pueden reproducir tanto en audio espacializado (tiene una lista de fuentes espacializadas) como sin espacializar.

    - ItemSpawnController.cs: Coloca los ingredientes en los puntos recibidos como argumento de manera aleatoria. Se asegura de que estén todos los ingredientes de la receta y rellena los demás huecos con ingredientes aleatorios.

- FeedbackController.cs: Se suscribe a los eventos de inicio y fin de ronda del manager y muestra a través de la pizarra blanca la receta y los resultados cuando estos se invocan.

RETOS DE ESTA SALA

Por último, durante el desarrollo de esta sala, se han encontrado varios problemas que han tenido que ser superados para llegar a la implementación final.

El primero, y el mayor de los problemas, ha sido la necesidad de tener que implementar las interacciones a partir de la SDK de Meta (primera vez que el equipo utilizaba este paquete) ya que su implementación de los snaps da la vuelta al entendimiento de interactor e interactable presentada en otros kits utilizados previamente por el equipo. Además, no cuentan con eventos, por lo que se tuvo que desarrollar una nueva forma de ver cuando un objeto se coloca en el snap para poder implementar mecánicas fundamentales en esta sala.
Por otro lado, también se ha tenido que trabajar con gran hincapié en el desarrollo de assets con IA, como los audios de instrucción, los cuales tuvieron que pasar por varias iteraciones y ediciones de audio (recortes, montaje, ecualizaciones, etc.) para llegar a su versión final.

Finalmente, un ligero problema con el que se tuvo que lidiar es la búsqueda de modelos para los ingredientes que fuesen coherentes con el resto de la estética, así como funcionales para el contexto de una aplicación XR. Esto probó ser más complicado de lo esperado.

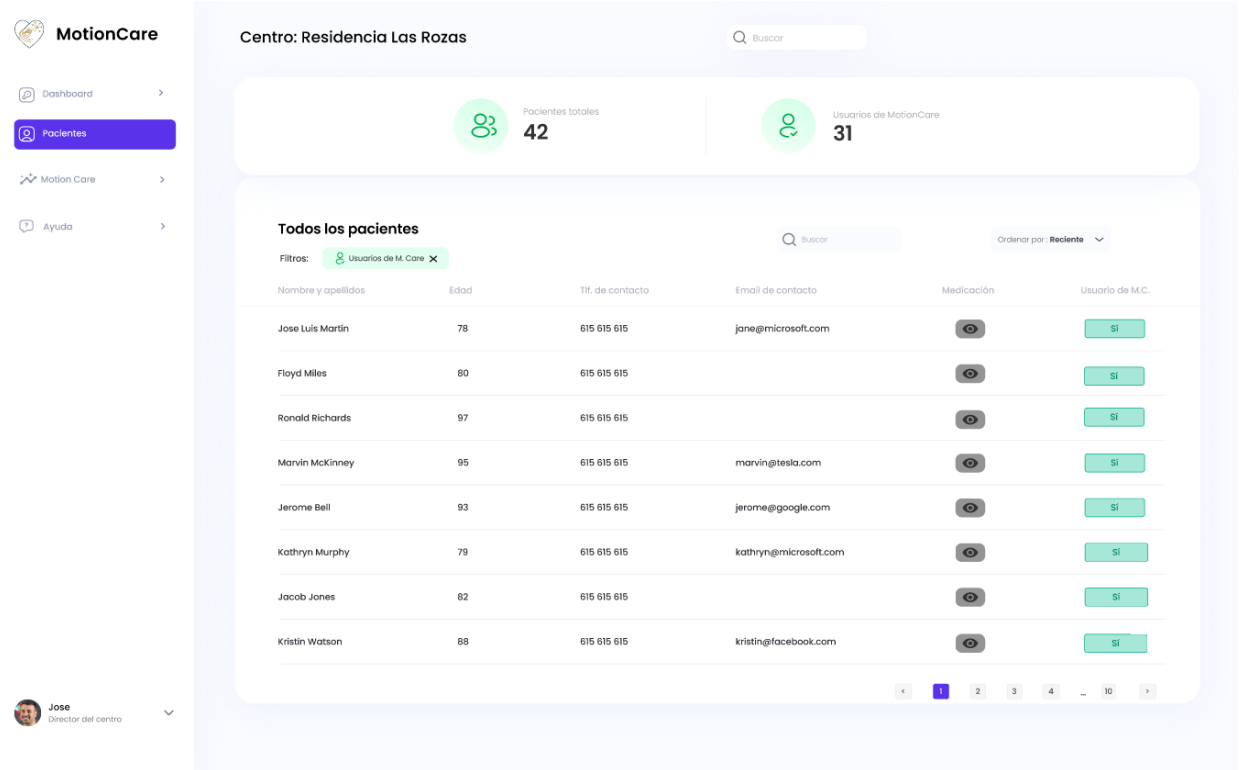
4. Conclusiones

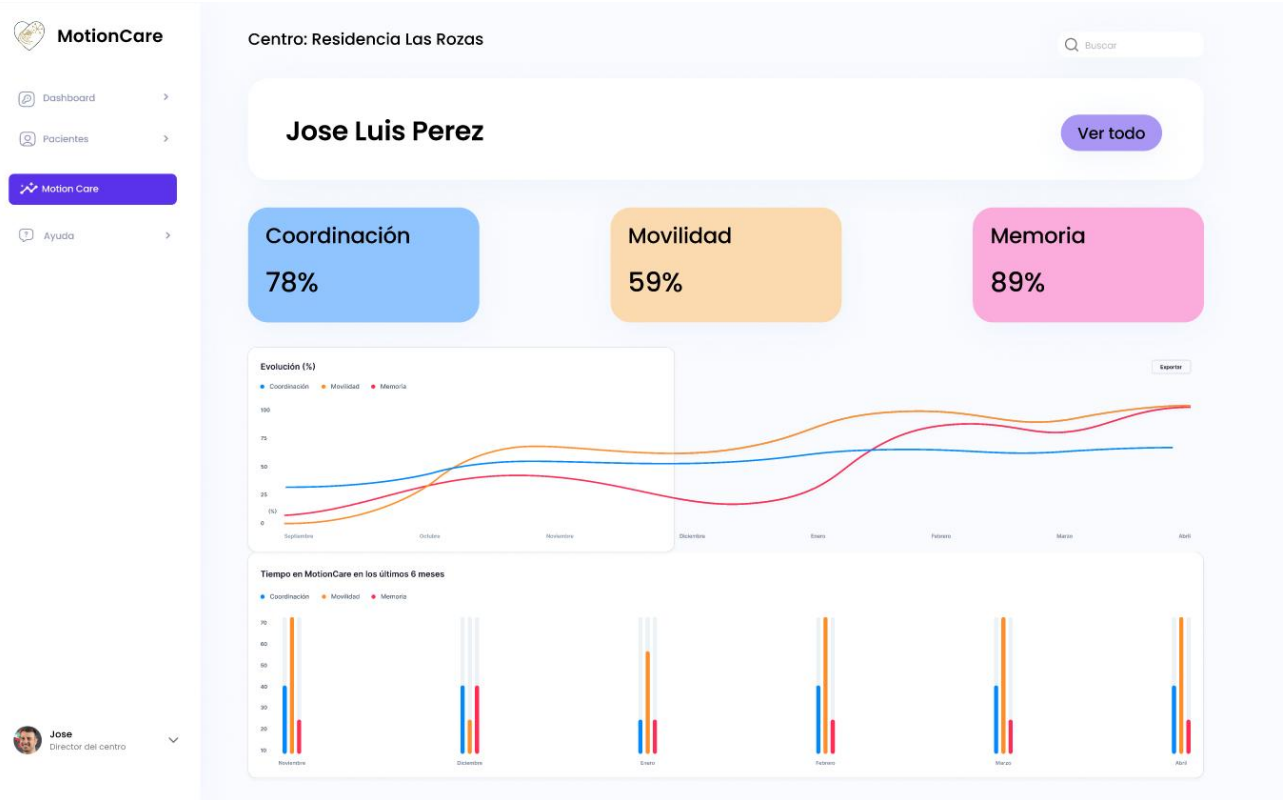
Motion Care surge como respuesta a una necesidad real: ofrecer a las personas de la tercera edad un entorno seguro, accesible y motivador donde puedan trabajar capacidades físicas y cognitivas sin depender de espacios especializados ni equipamiento complejo. A lo largo del desarrollo del proyecto, se ha demostrado que la realidad mixta puede convertirse en una herramienta eficaz para acercar el ejercicio y la rehabilitación a quienes más lo necesitan, transformando una habitación corriente en un espacio interactivo que estimula la memoria, la coordinación y la movilidad.
El diseño de las distintas estancias y minijuegos ha permitido crear una experiencia progresiva, clara y comprensible incluso para usuarios con poca familiaridad con la tecnología. Este enfoque ha sido clave para mantener la accesibilidad como eje central del proyecto, garantizando que cualquier persona, especialmente los mayores, pueda participar sin frustración y con una sensación constante de acompañamiento y guía.
Además, la integración de sistemas de recogida de métricas, como el DataManager y el ChronometerManager, aporta un valor añadido al permitir evaluar la evolución del usuario de forma objetiva. Esto abre la puerta a que Motion Care pueda convertirse en una herramienta útil no solo para el propio usuario, sino también para cuidadores y profesionales que deseen monitorizar su progreso y adaptar las actividades a sus necesidades.
En conjunto, Motion Care demuestra que la tecnología XR no tiene por qué ser una barrera, sino un puente hacia nuevas formas de bienestar. El proyecto sienta las bases de una plataforma que puede crecer, incorporar nuevos ejercicios y convertirse en un apoyo real para

mejorar la calidad de vida de las personas mayores, fomentando hábitos saludables, autonomía y, sobre todo, una experiencia de ejercicio que resulte cercana, divertida y significativa.

5. Trabajos futuros

Este proyecto se ha convertido en una fase inicial de uno mucho mayor. En el desarrollo de la idea se presentaron posibles trabajos futuros, tanto de Realidad Aumentada como de funcionalidades nuevas y extensiones a aplicaciones web.
Por un lado, se ideó la posibilidad de crear una plataforma Web para la gestión de cada paciente, así como sus puntuaciones, métricas y datos personales. De esta manera, los datos recogidos por la experiencia no quedarían en un simple archivo. La aplicación web básica incluiría una página para controlar el desarrollo del paciente, con gráficos y elementos visuales que acompañarían a las métricas. Para que esto no quedara en una simple idea, hemos desarrollado un pequeño mock-up utilizando Figma[3] – herramienta para el desarrollo de diseños web. En la imagen inferior se pueden ver las dos ventanas principales: la página de pacientes, y la ficha de uno de ellos.





Como se puede apreciar, tanto en la experiencia como en la aplicación web se ha priorizado la claridad visual, ya que una de las metas es llegar a un público que no tiene por qué estar familiarizado con la tecnología.

Entre las mejoras futuras, también se incluye la creación de nuevos juegos y la medición de distintas métricas. Además, se analizó la idea de poder personalizar la experiencia del paciente, es decir, dar la opción a la persona encargada de gestionar el juego que necesita el usuario. Todos los pacientes son diferentes, con distintas necesidades cada día, por ello, esta mejora añadiría un gran valor al proyecto, ya que el cuidado sería mucho más a medida. Otra ventaja de ello es que muchas veces los pacientes no tienen la capacidad de (o simplemente, no quieren) estar mucho tiempo haciendo una misma actividad, y mucho menos con una tecnología tan nueva como la realidad virtual. Esta mejora traería consigo una mayor aceptación por parte de los usuarios.

6. Bibliografía

[1] Frontera, Walter. (2018). Physical Activity and Rehabilitation in Elderly. 10.1007/978-3-319-57406-6_1.

[2] https://elevenlabs.io

[3] https://www.figma.com/proto/c8iVcXS6IfpSk5nd7xLjA1/MotionCare-basic-design?node-id=501-2&p=f&t=Gm2x8OCnMZhLqonk-0&scaling=scale-down&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=501%3A2